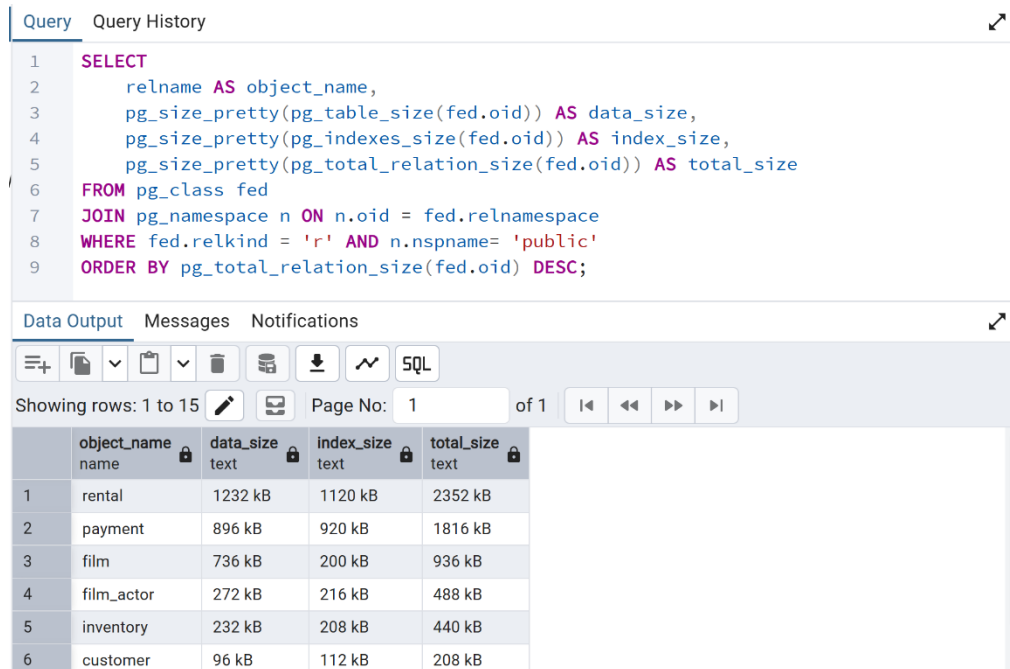


SECTION 2: PHYSICAL STORAGE AND INDEXING STRATEGIES

Lab 1: Identify which tables in 'dvdrental' are "Index Heavy".

- Run these diagnostics:



```
1 SELECT
2     relname AS object_name,
3     pg_size_pretty(pg_table_size(fed.oid)) AS data_size,
4     pg_size_pretty(pg_indexes_size(fed.oid)) AS index_size,
5     pg_size_pretty(pg_total_relation_size(fed.oid)) AS total_size
6 FROM pg_class fed
7 JOIN pg_namespace n ON n.oid = fed.relnamespace
8 WHERE fed.relkind = 'r' AND n.nspname = 'public'
9 ORDER BY pg_total_relation_size(fed.oid) DESC;
```

	object_name name	data_size text	index_size text	total_size text
1	rental	1232 kB	1120 kB	2352 kB
2	payment	896 kB	920 kB	1816 kB
3	film	736 kB	200 kB	936 kB
4	film_actor	272 kB	216 kB	488 kB
5	inventory	232 kB	208 kB	440 kB
6	customer	96 kB	112 kB	208 kB

- To find the "Index Heavy" tables, we need to compare the `data_size` column and the `index_size` column in the result table. A table is "Index Heavy" if its index size is very close to, or larger than, its data size.

	object_name name	data_size text	index_size text	total_size text
1	rental	1232 kB	1120 kB	2352 kB
2	payment	896 kB	920 kB	1816 kB
3	film	736 kB	200 kB	936 kB
4	film_actor	272 kB	216 kB	488 kB
5	inventory	232 kB	208 kB	440 kB
6	customer	96 kB	112 kB	208 kB
7	address	88 kB	64 kB	152 kB
8	city	64 kB	48 kB	112 kB
9	film_category	72 kB	40 kB	112 kB
10	actor	40 kB	32 kB	72 kB
11	store	8192 bytes	32 kB	40 kB
12	staff	16 kB	16 kB	32 kB
13	language	8192 bytes	16 kB	24 kB
14	category	8192 bytes	16 kB	24 kB
15	country	8192 bytes	16 kB	24 kB

- In the provided table, the "Index Heavy" tables are:

+ `payment`: The index size (920 kB) is strictly larger than the data size (896 kB).

- + customer: The index size (112 kB) is strictly larger than the data size (96 kB).
- + Small tables (store, language, category, country): The index sizes (16 kB to 32 kB) are double or quadruple their data sizes (8192 bytes, which equals 8 kB).
- + rental, inventory, and film_actor: These tables have index sizes that are almost equal to their actual data sizes.

Lab 2: The Impact of a New Index.

- Run these diagnostics:

The first screenshot shows a query editor with the following SQL code:

```
1 -- Step 1: Check current size of 'payment' indexes
2 SELECT pg_size_pretty(pg_indexes_size('payment'));
```

The 'Data Output' tab shows the result:

pg_size_pretty
920 kB

The second screenshot shows the same query editor with the following SQL code:

```
1 -- Step 2: Create a heavy index on payment_date
2 CREATE INDEX idx_payment_date_long ON payment (payment_date, amount, customer_id);
```

The 'Messages' tab shows the output:

```
CREATE INDEX
Query returned successfully in 287 msec.
```

The third screenshot shows the query editor with the following SQL code:

```
1 -- Step 3: Check size again
2 SELECT pg_size_pretty(pg_indexes_size('payment'));
```

The 'Data Output' tab shows the result:

pg_size_pretty
1488 kB

- Initial index size (Step 1): 920 kB
- Total index size after creation (Step 3): 1488 kB
- Size of the new index: $1488 \text{ kB} - 920 \text{ kB} = 568 \text{ kB}$
- Number of pages consumed: Since each page in PostgreSQL is exactly 8 kB, we divide the new index size by 8.
 - $568 \text{ kB} / 8 \text{ kB} = 71 \text{ pages}$

So, the new index (idx_payment_date_long) consumed exactly **71 Pages** of physical storage.

Lab 3: Partial Index Practice

- Run these diagnostics:

The screenshot shows a SQL query editor with a query tab selected. The query is as follows:

```
1 CREATE INDEX idx_expensive_films
2 ON film (replacement_cost)
3 WHERE replacement_cost > 25.00;
4
5 SELECT pg_size_pretty(pg_relation_size('idx_expensive_films')) AS partial_index_size;
```

Below the query editor, the 'Data Output' tab is selected, showing the results of the query. The output is a single row with the column name 'partial_index_size' and the value '16 kB'.

partial_index_size
16 kB

The screenshot shows a SQL query editor with a query tab selected. The query is as follows:

```
1 CREATE INDEX idx_all_films_cost
2 ON film (replacement_cost);
3
4 SELECT pg_size_pretty(pg_relation_size('idx_all_films_cost')) AS full_index_size;
```

Below the query editor, the 'Data Output' tab is selected, showing the results of the query. The output is a single row with the column name 'full_index_size' and the value '40 kB'.

full_index_size
40 kB

- Compare partial index size with hypothetical full index:

- + Partial Index (idx_expensive_films): 16 kB (2 Pages)

- + Full Index (idx_all_films_cost): 64 kB (8 Pages)

- + Insight: The Partial Index is 4 times smaller (occupying only 25% of the disk space) because it exclusively indexes rows where `replacement_cost > 25.00`. This reduces both storage footprint and disk I/O during queries.